

Pre entrega N°1

Guía orientativa para su desarrollo

Introducción:

Este documento es una guía paso a paso para ayudarte a completar los requisitos de la "Primera Pre-Entrega: Configuración Inicial del Proyecto y Repositorio". Seguí estas instrucciones para asegurarte de que tu proyecto comience con una base sólida y organizada.

1. Creación del Proyecto LibGDX con gdx-liftoff

LibGDX utiliza una herramienta de configuración para generar la estructura básica de tu proyecto. **Liftoff** es la herramienta modern

a para generar la estructura de tu proyecto LibGDX.

- **Descargar gdx-liftoff:**
 - Anda a la página de [Releases de gdx-liftoff en GitHub](#).
 - Descarga el archivo .jar más reciente (ej. gdx-liftoff-VERSION.jar).
- **Requisito de Java:**
 - Asegurate de tener instalado **Java 17 o superior** para ejecutar Liftoff y construir el proyecto. Podes verificar tu versión con `java -version` en la terminal.
- **Ejecutar gdx-liftoff:**
 - La forma más común es hacer doble clic en el archivo .jar descargado.
 - Si eso no funciona, puedes abrir una terminal o símbolo del sistema, navegar a la carpeta donde descargaste el archivo y ejecutar:

```
java -jar gdx-liftoff-VERSION.jar
```

(Reemplaza VERSION con la versión que descargaste). En Linux, puede que necesites dar permisos de ejecución al archivo.

Configurar tu Proyecto en Liftoff:

1. Basic Settings (Ajustes Básicos):

- **Project Name:** El nombre de tu aplicación/juego (ej. MiSuperJuego).
- **Package Name:** El paquete Java base para tu código (ej. com.miempresa.mijuego).
- **Main Class Name:** El nombre de tu clase principal del juego (ej. MiJuegoPrincipal).

2. Project Options (Opciones del Proyecto): Haz clic en el botón "Project Options" o similar para acceder a las personalizaciones. Esta sección generalmente se divide en (al menos) dos pestañas o pasos principales:

- **Pestaña/Paso 1: Add-Ons (Complementos / Opciones Principales):**
 - **Platforms:** Selecciona las plataformas destino. Core es obligatorio. Desktop (LWJGL3) es altamente recomendado para pruebas. Marca también HTML,

Android, iOS según tus necesidades. (Recordatorio: iOS requiere macOS y Xcode). En la mayoría de los casos con Core y Desktop alcanza.

- **Languages:** Además de Java (que es la base), se puede añadir soporte para otros lenguajes como Kotlin, Groovy o Scala. Recomendando no agregar esto por ahora.
- **Extensions (Official):** Acá puedes seleccionar extensiones *oficiales* de LibGDX que forman parte del framework principal, como Box2D (físicas 2D), Ashley (Entity Component System), Freetype (fuentes TTF), etc. Elegí las que preveas usar, ya que añadirlos más tarde es bastante laborioso.
- **Template:** Elegí una plantilla inicial para tu proyecto. Classic es similar al antiguo setup. Otras plantillas ofrecen estructuras básicas para diferentes tipos de juegos o características. Classic es un buen punto de partida.

- **Pestaña/Paso 2: Third-Party (Extensiones de Terceros):**

- Acá encontrarás una lista de bibliotecas populares desarrolladas por la *comunidad* de LibGDX, no por el equipo oficial principal. Ejemplos incluyen VisUI (widgets de interfaz gráfica), ktx (utilidades para Kotlin), ShapeDrawer (dibujo de formas mejorado), etc.
- Puedes buscar por nombre y seleccionar las que necesites. Si no estás seguro, puedes añadir las más tarde modificando el archivo build.gradle. Para esta primera entrega, probablemente no necesites añadir ninguna extensión de terceros.

3. Advanced Settings (Opciones Avanzadas - Opcional): Puedes ajustar versiones específicas de LibGDX, Gradle, etc. Normalmente lo mejor sería dejar los valores por defecto. En Project path se debe seleccionar la ruta donde quieras generar el proyecto.

4. Generate (Generar): Haz clic en "Generate". Liftoff descargará las dependencias y creará la estructura del proyecto en la carpeta de destino que especificaste.

Importar en tu IDE

Si bien trabajamos con Eclipse para fines educativos, recomendamos que para el proyecto usen IntelliJ.

- **IntelliJ IDEA / Android Studio:**

- Anda a "File" > "Open..." (o "Open" en la pantalla de bienvenida).
- Navega hasta la carpeta del proyecto que Liftoff acaba de crear.
- Selecciona el archivo build.gradle (o la carpeta que lo contiene) y elegí "Open as Project". El IDE debería reconocerlo como un proyecto Gradle.

- **Eclipse:**

- Anda a "File" > "Import..."
- Selecciona "Gradle" > "Existing Gradle Project".
- Navega hasta la carpeta raíz del proyecto generado y seguí los pasos del asistente.

Verificar Ejecución:

- Una vez importado (*puede tardar un rato en importar el proyecto, aunque veas que ya se importó debajo de todo puede aparecer un mensaje que dice "importing x Gradle Project". Hasta que este mensaje no desaparezca el proyecto no estará importado*), busca el módulo/subproyecto de escritorio. **Nota:** Con Liftoff, en general se llama lwjgl3 en lugar de desktop.
- Dentro de ese módulo, busca la clase lanzadora (ej. Lwjgl3Launcher.java) y ejecutala (Run As > Java Application o similar).
- Deberías ver la pantalla de ejemplo de la plantilla que elegiste (o la imagen por defecto de LibGDX si no elegiste plantilla). Es posible que necesites configurar el "Working Directory" en la configuración de ejecución para que apunte a la carpeta assets/ del proyecto core/ si las imágenes no cargan.

2. Configuración del Repositorio en GitHub

GitHub es una plataforma para alojar y gestionar tus proyectos usando Git, un sistema de control de versiones.

- **Paso 2.1: Crear una Cuenta en GitHub (si no tenes una)**
 1. Anda a github.com y seguí el proceso de registro ("Sign up"). Es gratuito para repositorios públicos y privados.
- **Paso 2.2: Instalar Git (si no lo tenes)**
 1. Git es el sistema de control de versiones que se ejecuta en tu computadora. GitHub es la plataforma web donde alojas tus repositorios Git. Necesitas instalar Git localmente.
 2. Descarga: Anda al sitio web oficial de Git: <https://git-scm.com/downloads>. El sitio generalmente detecta tu sistema operativo y te ofrece la descarga adecuada.
 3. Instalación:
 - Windows: Ejecuta el instalador descargado. Durante la instalación, te hará varias preguntas. Para empezar, puedes dejar las opciones por defecto seleccionadas. Asegurate de que la opción "Git Bash Here" (o similar) esté seleccionada si quieres una terminal estilo Linux integrada, y que Git se añada al PATH si quieres usarlo desde el cmd o PowerShell estándar. La opción recomendada para el editor suele ser Vim (difícil para principiantes). Si usas IntelliJ o Eclipse, lo más probable es que *casi nunca* necesites este editor porque harás los commits usando la interfaz gráfica del IDE o usando siempre `git commit -m "Tu mensaje corto"` en la terminal. Por lo tanto, puedes elegir **Notepad**, **Notepad++** o **VS Code** si lo tenés, o incluso dejar **Vim** (el predeterminado a menudo). Para el nombre de la rama inicial, la opción por defecto ahora suele ser main.
 - macOS: La forma más fácil suele ser instalar las "Xcode Command Line Tools". Abrió la Terminal (Aplicaciones > Utilidades > Terminal) y escribe `git --version`. Si no lo tenes, te ofrecerá instalar las herramientas de línea de comandos. Aceptalo. Alternativamente, podés descargar el instalador desde el sitio web de Git o usar un gestor de paquetes como Homebrew (`brew install git`).
 - Linux (Debian/Ubuntu): Abrió una terminal y ejecuta: `sudo apt update && sudo apt install git`
 - Linux (Fedora): Abrió una terminal y ejecuta: `sudo dnf install git`

4. **Verificar Instalación:** Abrió una nueva terminal (o Git Bash en Windows) y escribí `git -version`. Debería mostrarte la versión de Git instalada.
5. **Configuración Inicial (Importante):** Después de instalar Git, configuralo con tu nombre y correo electrónico. Estos datos se asociarán a tus commits. Abrió una terminal y ejecuta:

```
git config --global user.name "Tu Nombre Completo"
git config --global user.email "tu_correo@ejemplo.com"
```

(Usa el mismo correo electrónico que usaste para registrarte en GitHub).

- **Paso 2.3: Crear un Nuevo Repositorio en GitHub**

1. **Inicia Sesión en GitHub:** Anda a github.com y accede con tu cuenta.
2. **Botón "New":** En tu página principal de GitHub (Dashboard), busca un botón verde "New" o hace clic en el signo "+" en la esquina superior derecha y selecciona "New repository".
3. **Formulario de Creación:**
 - **Owner:** Asegurate de que esté seleccionado tu nombre de usuario.
 - **Repository name:** Elegí un nombre para tu repositorio. Debe ser único dentro de tu cuenta. Usa nombres cortos, descriptivos, con guiones bajos o medios si es necesario (ej. proyecto-final-tp, mi-juego-libgdx).
 - **Description (Optional):** Escribe una breve descripción de lo que tratará el proyecto. Ayuda a otros (y a tu yo futuro) a entender de qué se trata.
 - **Public / Private:**
 - **Public:** Cualquiera en internet puede ver tu repositorio, pero solo vos (y tus colaboradores) pueden hacer cambios. Ideal para proyectos open source. En este caso vamos a necesitar que sea público, ya que de otra manera GitHub no permitirá agregar una wiki. Asegurate de añadir a los profesores como colaboradores.
 - **Initialize this repository with:** (IMPORTANTE: **NO** marques ninguna de estas opciones por ahora):
 - Add a README file: Lo crearemos nosotros localmente.
 - Add .gitignore: Lo crearemos nosotros localmente (o usaremos el generado por Liftoff).
 - Choose a license: Podes agregar una licencia más tarde si queres.
4. **Create Repository:** Hace clic en el botón verde "Create repository".

- **Paso 2.4: Subir tu Proyecto LibGDX Existente al Nuevo Repositorio**

- Después de crear el repositorio en GitHub, te mostrará una página con instrucciones. Busca la sección que dice **"...or push an existing repository from the command line"**. Estos son los comandos que usarás ahora:
- 1. **Abrir una Terminal o Git Bash:** Navega hasta la carpeta raíz de tu proyecto LibGDX (la que creaste con gdx-liftoff).

2. Inicializar Git en tu Proyecto Local (si no lo hiciste antes):

```
git init -b main
```

3. Añadir todos los archivos del proyecto al "staging area":

- Primero, asegurate de que el archivo .gitignore generado por Liftoff existe en la carpeta raíz. Si no existe, crea uno básico.

```
git add .
```

4. Realizá el primer commit

```
git commit -m "Configuración inicial del proyecto con LibGDX Liftoff "
```

5. Conectar tu Repositorio Local con el Remoto: Copia la línea de GitHub que empieza con git remote add origin ...:

```
git remote add origin https://github.com/TU_USUARIO/TU_REPOSITORIO.git
```

(Reemplaza TU_USUARIO y TU_REPOSITORIO con los tuyos). Pega y ejecuta esta línea en tu terminal.

6. Subir tus Cambios (Commit) al Repositorio Remoto: Copia la línea de GitHub que empieza con git push -u origin ...:

```
git push -u origin main
```

Pega y ejecuta esta línea. Es posible que te pida autenticarte con GitHub (usuario/contraseña o mediante un token/SSH si lo tienes configurado).

7. ¡Listo! Refresca la página de tu repositorio en GitHub. Ahora deberías ver los archivos de tu proyecto.

- **Paso 2.5: (Si trabajas en grupo o necesitas dar acceso al profesor) Añadir Colaboradores:**
 1. En la página de tu repositorio en GitHub, anda a la pestaña "Settings".
 2. En el menú lateral izquierdo, selecciona "Collaborators and teams" (o simplemente "Collaborators").
 3. Hace clic en el botón "Add people".
 4. Busca a tus compañeros o al profesor por su nombre de usuario de GitHub (jasinski1988), correo electrónico o nombre completo. Selecciónalos y haz clic en "Add [username] to this repository".
 5. Ellos recibirán una invitación por correo electrónico que deberán aceptar.

3. Creación del Archivo README.md

Este archivo es la carta de presentación del proyecto en GitHub. Debe ser claro y conciso.

- **Contenido Sugerido para el README.md:**
 - **Nombre del Proyecto:** Un título claro y descriptivo.
 - **Integrantes del Grupo:** Listar los nombres de los participantes.
 - **Descripción Corta del Juego:** Un párrafo breve explicando la idea general del videojuego (la misma que fue aprobada o está en proceso de aprobación).
 - **Tecnologías Utilizadas:** Mencionar LibGDX y las plataformas objetivo (ej. Escritorio, Web, Móvil).
 - **Cómo Compilar y Ejecutar:** Instrucciones básicas para que cualquier persona (incluyéndote) pueda clonar el repositorio y correr el juego.
 - **Estado Actual del Proyecto:** Para esta primera entrega, podría ser "Configuración inicial y estructura del proyecto".

4. Creación del Archivo de Cambios (CHANGELOG.md o changes.md):

- Este archivo es crucial para llevar un registro de las modificaciones importantes que se realizan en el proyecto a lo largo del tiempo.
- **Contenido y Formato:**
 - Se debe registrar cada conjunto de cambios significativos con la fecha y una breve descripción.
 - Generalmente se organiza en orden cronológico inverso (lo más nuevo arriba).
 - Ejemplo de formato (Keep a Changelog es un estándar popular):

```
## [Unreleased]
### Added
- Nueva funcionalidad X.

## [1.0.0] - 2023-10-26
### Added
- Implementación inicial del movimiento del personaje. [cite: 10]
- Creación del mapa básico. [cite: 9]
### Changed
- Mejora en la textura del personaje.
### Fixed
- Corregido error al cargar el juego.
```

Para esta primera entrega, podrían inicializarlo con la creación del proyecto.

5. Configuración de la Wiki de GitHub para el Proyecto

La Wiki es una sección dentro de tu repositorio de GitHub ideal para documentación más extensa y organizada, como la propuesta formal de tu proyecto. Permite mantener un historial de cambios fácilmente.

- **Paso 5.1: Acceder a la Wiki:**

- Anda a la página principal de tu repositorio en GitHub.
- Busca la pestaña **"Wiki"** en la barra de navegación superior (junto a "Code", "Issues", "Pull requests", etc.). Haz clic en ella.
 - **Nota:** Si no ves la pestaña "Wiki", puede que esté desactivada. Anda a "Settings" (Configuración) > "General", baja hasta la sección "Features" (Características) y asegurate de que la casilla "Wiki" esté marcada. Luego vuelve a la página principal del repositorio y la pestaña Wiki debería aparecer.

- **Paso 5.2: Crear la Primera Página (o la Página de la Propuesta):**

- Si es la primera vez que entras a la Wiki de este repositorio, verás un mensaje de bienvenida y un botón grande verde que dice **"Create the first page"**. Hace clic en él.
- Si ya existen otras páginas en la Wiki (por ejemplo, la página Home por defecto), busca un botón que diga **"New Page"** (Nueva Página) en la esquina superior derecha del área de la Wiki. Hace clic en él.

- **Paso 5.3: Editar el Contenido de la Página:**

- **Page Title (Título de la Página):** Escribí un título claro y descriptivo para esta página. Por ejemplo: Propuesta del Proyecto - [Nombre del Juego]. La página principal por defecto suele llamarse Home. Podes usar esa o crear una nueva específicamente para la propuesta.
- **Content Format (Formato del Contenido):** Asegurate de que esté seleccionado **"Markdown"**. Este es el lenguaje de formato que hemos estado usando para README.md y CHANGELOG.md. Permite añadir títulos, listas, negritas, enlaces, etc., de forma sencilla.
- **Content Area (Área de Contenido):** Este es el editor principal. Acá es donde debes **pegar o escribir el texto completo de tu propuesta formal del proyecto**. Utiliza la sintaxis de Markdown para estructurarla:
 - Usa # para títulos principales (ej. # Propuesta del Juego: Space Invaders Remake).
 - Usa ## para subtítulos (ej. ## 1. Concepto General).
 - Usa ### para sub-subtítulos.
 - Usa * o - al inicio de una línea para crear listas con viñetas.
 - Usa ****texto**** para poner texto en **negrita**.
 - Usa **texto** para poner texto en *cursiva*.
 - Usa [texto del enlace](URL) para crear hipervínculos.
- **Edit summary (Optional) (Resumen de la Edición - Opcional):** Debajo del editor principal, hay un campo opcional para describir brevemente los cambios que estás haciendo en esta edición (ej. "Versión inicial de la propuesta", "Añadida sección de mecánicas", "Corrección de errores tipográficos"). Es una buena práctica para llevar un registro.

- **Paso 5.4: Guardar la Página:**
 - Una vez que hayas terminado de añadir y formatear el contenido de tu propuesta, hace clic en el botón verde "**Save Page**" (Guardar Página).
 - Tu página de la Wiki con la propuesta ya está creada y visible.
- **Paso 5.5: Navegar y Obtener el Enlace:**
 - En la parte derecha de la pantalla de la Wiki, verás una barra lateral llamada "Pages" que lista todas las páginas creadas. Podes hacer clic en los nombres para navegar entre ellas.
 - Para obtener el enlace directo a tu página de propuesta, simplemente navega a esa página y **copia la URL completa** que aparece en la barra de direcciones de tu navegador web.
- **Paso 5.6: Enlazar desde el README.md:**
 - Vuelve a editar tu archivo README.md (ya sea localmente y haciendo git push, o directamente en la interfaz de GitHub).
 - En la sección correspondiente (ver Punto 3 del tutorial), asegurate de que el enlace a la Wiki apunte a la URL que copiaste en el paso anterior. Ejemplo en Markdown:

```
**Enlace a la Wiki del Proyecto (Propuesta Detallada):**
[Ver la Propuesta Completa del Proyecto
aquí](https://github.com/TU_USUARIO/TU_REPOSITORIO/wiki/Propuesta-del-
Proyecto----Nombre-del-Juego)
```

(Reemplaza con tu URL real).

- **Paso 5.7: Ver el Historial (Para el Futuro):**
 - Si haces clic en una página de la Wiki, verás una pestaña o botón llamado "**Page History**" (Historial de la Página) en la parte superior derecha. Al hacer clic ahí, podrás ver todas las versiones anteriores de esa página, quién hizo los cambios y cuándo. Esto es muy útil para ver cómo evoluciona la propuesta.

6. El Archivo .gitignore

Este archivo le dice a Git qué archivos o carpetas ignorar y no subir al repositorio.

- **Propósito:** Evita subir archivos generados por el IDE, archivos de compilación, dependencias locales, etc., que no son parte del código fuente esencial o que pueden variar entre computadoras.
- **Proyectos LibGDX:** La herramienta gdx-liftoff usualmente genera un archivo .gitignore bastante completo en la raíz del proyecto. **Revísalo.**
- **Qué debería incluir (generalmente ya está en el autogenerado):**

```
# Archivos de compilación
.gradle
build/
*/build/

# Archivos de IDEs
.idea/
*.iml
*.ipr
*.iws
.project
.classpath
.settings/
*.nbproject/

# Archivos locales
local.properties

# Archivos de sistema operativo
.DS_Store
Thumbs.db
```

- **Acción:** Si el archivo `.gitignore` fue generado automáticamente y parece correcto, simplemente asegúrate de que esté añadido y commiteado en tu repositorio (normalmente se hace en el primer commit). Si necesitas modificarlo:

1. Edita el archivo `.gitignore`.
2. Guarda los cambios.

```
git add .gitignore
git commit -m "Docs: Actualización de .gitignore"
git push origin main
```

7. Pautas para Trabajar con GitHub y Documentación:

- **Commits Frecuentes y Atómicos:**
 - **Commit:** Es como guardar una instantánea de tus cambios.
 - Hace commits pequeños y enfocados en una sola tarea o arreglo. (Ej. "Implementada la función de salto" en lugar de "Muchos cambios").
 - **Mensajes de Commit Claros:** Escribe mensajes descriptivos que expliquen *qué* cambiaste y *por qué*. Un formato común es:
 - Feat: Agregado de nueva funcionalidad de salto (Feature)
 - Fix: Corregido error de colisión con plataformas (Bug fix)
 - Docs: Actualizado instrucciones en README (Documentation)
 - Style: Mejorado formato de código en Player.java (Code style)
 - Refactor: Optimizado el método de movimiento (Code refactoring)
 - Test: Añadido pruebas para el sistema de puntuación (Tests)

- **Ciclo Básico de Trabajo:**

1. **git pull origin main:** (O git pull origin NOMBRE_DE_TU_RAMA) Siempre antes de empezar a trabajar, para traer los últimos cambios del repositorio remoto y evitar conflictos, especialmente si trabajas en grupo.
2. **Trabaja en tus archivos:** Modifica, crea, elimina archivos.
3. **git status:** Para ver qué archivos has modificado.
4. **git add NOMBREDELARCHIVO_O_CARPETA** o **git add .:** Para añadir tus cambios al "staging area" (prepararlos para el commit).
5. **git commit -m "Tu mensaje descriptivo":** Para guardar los cambios localmente.
6. **git push origin main:** (O git push origin NOMBRE_DE_TU_RAMA) Para subir tus commits al repositorio remoto en GitHub.

- **Ramas (Branches) - (Buena Práctica, especialmente para después):**

- **main (o master):** Debería contener siempre la versión estable y funcional de tu proyecto.
- **develop:** Una rama donde integras funcionalidades completadas.
- **Ramas de Funcionalidad (feature/nombre-funcionalidad):** Para cada nueva característica grande, crea una nueva rama (ej. feature/menu-principal). Trabaja en ella y, una vez terminada y probada, la fusionas (merge) con develop.
- Para esta primera entrega, si trabajas solo, puedes usar main directamente. Si están en grupo, considera usar una rama develop desde el inicio.
- Crear una rama: git branch nombre-nueva-rama
- Cambiar a una rama: git checkout nombre-nueva-rama
- Crear y cambiar en un paso: git checkout -b nombre-nueva-rama

¡Listo! Siguiendo estos pasos, deberías poder completar todos los requisitos de la primera pre-entrega. Recuerda consultar cualquier duda con el profesor. ¡Mucho éxito con tu proyecto!